

Documento técnico del proyecto FondoXYZ

Sistema para gestionar sedes, apartamentos, disponibilidad, tarifas, reservas, pagos y asociados.

1. Arquitectura utilizada en el proyecto

El proyecto es una aplicación ASP.NET Core 8 con enfoque de API REST y patrón por capas. La aplicación expone controladores HTTP en la ruta /api, delega la lógica de negocio a servicios inyectados por dependencia y usa Entity Framework Core como capa de acceso a SQL Server.

- Aplicación web/API monolítica: el backend, las reglas de negocio y el acceso a datos conviven en un solo proyecto .NET.
- Controladores API: reciben solicitudes, validan cuerpos nulos, traducen errores de negocio a respuestas HTTP y devuelven DTOs.
- Servicios de dominio/aplicación: concentran reglas de asociados, autenticación, sitios, disponibilidad, tarifas, reservas y pagos.
- Persistencia: FondoXYZDbContext mapea entidades a tablas SQL Server y también ejecuta procedimientos almacenados para disponibilidad y tarifas.
- Seguridad: la autenticación se realiza con JWT Bearer. Por defecto los controladores quedan protegidos mediante AuthorizeFilter, excepto /api/auth/login.

Componente	Responsabilidad
Program.cs	Configura DI, Entity Framework Core, autenticacion JWT, autorizacion global, controladores y pipeline HTTP.
Controllers/Api	Expone endpoints REST para autenticacion, asociados, sitios, disponibilidad, tarifas, reservas y pagos.
Services	Implementa reglas de negocio, validaciones, calculos, transacciones y llamadas a procedimientos almacenados.
Models/DTOs	Define contratos de entrada y salida de la API.
Data/Entities	Define entidades persistentes que representan tablas relacionales.
Data/FondoXYZDbContext.cs	Configura DbSet, relaciones, precision decimal, longitudes y mapeos de entidades.

Database/FondoXYZ.sql	Crea la base FondoXYZ, tablas, restricciones y datos semilla.
-----------------------	---

2. Estructura de capas implementada

La solución no separa las capas en proyectos independientes, pero si aplica separación lógica por carpetas y responsabilidades.

Capa	Descripcion
Presentacion/API	Controladores ASP.NET Core: AuthController, AsociadosController, SitiosController, DisponibilidadController, TarifasController y ReservasController.
Aplicación/Servicios	Interfaces y clases de servicio: IAuthService, IAsociadoService, ISitioService, IDisponibilidadService, ITarifaService, IReservaService e IPagoService.
Contratos DTO	Objetos de transporte para requests y responses, evitando exponer directamente entidades de base de datos.
Dominio/Persistencia	Entidades como Asociado, Sitio, UnidadAlojamiento, Tarifa, Reserva, Pago y relaciones asociadas.
Infraestructura de datos	FondoXYZDbContext, SQL Server, script Database/FondoXYZ.sql y procedimientos almacenados invocados desde servicios.

El flujo típico es: cliente HTTP -> controlador -> interfaz de servicio -> servicio concreto -> DbContext/procedimiento almacenado -> DTO de respuesta.

3. Modelo de base de datos relacional diseñado

El modelo relacional está centrado en sitios de alojamiento, unidades disponibles, tarifas, reservas y pagos. El script SQL crea llaves primarias, llaves foráneas, restricciones CHECK, restricciones UNIQUE y datos semilla para operar el sistema.

Grupo de tablas	Tablas y proposito
Catalogos base	Region, CategoriaTarifa, Temporada, CalendarioEspecial, TipoServicio y Asociado. Definen ubicaciones, categorias, temporadas, fechas especiales, servicios y usuarios asociados.
Sitios y unidades	Sitio, BloqueAlojamiento, ServicioSitio y UnidadAlojamiento. Modelan sedes recreativas, apartamentos, bloques, servicios disponibles y unidades reservables.
Tarifas	Tarifa almacena conceptos como NocheBase, NochePorPersonas, NocheApartamento, PersonaAdicional, VisitaDiaAcompanante y ServicioAdicional, con vigencias, temporada, categoria y filtros de aplicación.

Reservas	Reserva es la cabecera; ReservaUnidad registra unidades y subtotales por rango; ReservaAcompante registra acompañantes cobrados; ReservaServicio registra servicios adicionales; AuditoriaTarifa conserva el detalle calculado.
Disponibilidad y pagos	BloqueoDisponibilidad impide reservar unidades en rangos determinados; Pago registra metodo, monto, estado y fecha asociada a una reserva.

Relaciones principales

- Region 1:N Sitio. Cada sitio pertenece a una region.
- Sitio 1:N BloqueoAlojamiento, ServicioSitio y UnidadAlojamiento.
- CategoriaTarifa 1:N UnidadAlojamiento y 1:N Tarifa.
- Temporada 1:N Tarifa. La tarifa puede quedar asociada a baja o alta temporada.
- Asociado 1:N Reserva. Cada reserva pertenece a un asociado activo.
- Reserva 1:N ReservaUnidad, ReservaAcompante, ReservaServicio, AuditoriaTarifa y Pago.
- UnidadAlojamiento 1:N ReservaUnidad y 1:N BloqueoDisponibilidad.

Restricciones relevantes

- Sitio.TipoSitio solo permite SedeRecreativa o Apartamento.
- Reserva.TipoReserva solo permite Alojamiento o VisitaDia.
- Reserva.Estado solo permite Borrador, PendientePago, Confirmada, Completada, Cancelada o Expirada.
- Pago.Estado solo permite Iniciado, Aprobado, Rechazado o Reembolsado.
- Existen unicidades para códigos, correos, documentos, numero de asociado y Código de reserva.

4. Lógica para manejo de disponibilidad y reservas

La disponibilidad se consulta antes de crear reservas de alojamiento. El servicio valida fechas, sitio activo y capacidad, y delega la selección de unidades disponibles al procedimiento SP_ConsultarHabitacionesDisponibles.

Disponibilidad

- fechaEntrada y fechaSalida son obligatorias; para alojamiento la salida debe ser posterior a la entrada.
- Opcionalmente se filtra por sitioId y numeroPersonas. Si se consulta una unidad específica, se valida que este activa y que su sitio también este activo.
- El resultado devuelve unidades disponibles con datos de sitio, ciudad, capacidad, numero de habitaciones internas y categoría tarifaria.
- La lógica declarada en controladores indica que se excluyen unidades ocupadas por reservas activas y bloqueos de disponibilidad.

Creación de reservas de alojamiento

- Se valida asociado activo, sitio activo, tipo de reserva, fechas, cantidad de personas y unidades seleccionadas.
- Las unidades deben existir, estar activas y pertenecer al sitio informado.
- La capacidad combinada de las unidades debe cubrir el número total de personas; además se valida que una distribución promedio por unidad no exceda capacidades individuales.
- Se verifica disponibilidad contra el servicio de disponibilidad antes de guardar.
- Las unidades se agrupan por CategoriaTarifaId para calcular tarifas coherentes por tipo de unidad.
- El cálculo se realiza con SP_CalcularTarifaReserva y se guarda detalle en AuditoriaTarifa.
- La reserva se guarda en estado PendientePago dentro de una transacción junto con unidades, servicios y auditoria.

Creación de visitas de día

- Solo aplica a sitios de tipo SedeRecreativa.
- La fecha de entrada y salida debe ser la misma.
- Admite entre 5 y 10 personas. Las primeras cuatro personas no generan acompañante de pago; del quinto en adelante se registra acompañante y tarifa.
- Usa la tarifa activa con TipoConcepto VisitaDiaAcompañante.

Pagos y confirmación

- El pago debe tener monto mayor que cero y método de pago.
- Para pagos no reembolsados, el monto debe coincidir con el total de la reserva.
- Un pago Aprobado cambia la reserva a Confirmada y registra FechaConfirmacion.
- Pagos Iniciado o Rechazado mantienen la reserva en PendientePago.
- No se permite pagar reservas Canceladas o Expiradas ni aprobar una reserva ya confirmada.

5. Procedimientos almacenados desarrollados o utilizados

El Código consume tres procedimientos almacenados en SQL Server.

Procedimiento	Uso esperado
SP_ConsultarHabitacionesDisponibles	Recibe FechaEntrada, FechaSalida, Sitiold opcional y NumeroPersonas opcional. Debe devolver unidades activas disponibles, excluyendo reservas y bloqueos que se crucen con el rango solicitado.
SP_ConsultarTarifas	Recibe Sitiold, UnidadAlojamientold, Temporadald y NumeroPersonas opcionales segun consulta. Devuelve tarifas vigentes con informacion de sitio, categoria, temporada, precios, reglas de exclusion y vigencias.
SP_CalcularTarifaReserva	Recibe sitio, rango de fechas, numero de personas, numero de unidades, unidad o habitaciones internas y temporada opcional. Devuelve un primer resultado resumen con total y un segundo resultado con detalle diario/conceptual para auditoria.

Observación técnica

Los servicios DisponibilidadService y TarifaService capturan SQLException de severidad de usuario y devuelven el mensaje como ArgumentException, lo que permite que los controladores respondan 400 Bad Request con mensajes claros.

6. Tecnologías, librerías y componentes utilizados

tecnología	Uso en el proyecto
.NET 8 / C#	Plataforma principal del backend, con nullable reference types e implicit usings habilitados.
ASP.NET Core Web	Hosting, pipeline HTTP, controladores, filtros de autorizacion y respuestas REST.
Entity Framework Core SQL Server 8.0.11	Mapeo objeto-relacional, DbContext, consultas LINQ, transacciones y ejecucion de SQL/procedimientos.
Microsoft.AspNetCore.Authentication.JwtBearer 8.0.11	Validacion de tokens JWT en endpoints protegidos.
Microsoft.IdentityModel.Tokens	Configuracion de issuer, audience, key y firma HMAC SHA-256 para JWT.
Microsoft.AspNetCore.Identity PasswordHasher	Hash y verificacion de claves de asociados.
SQL Server	Motor relacional para tablas, restricciones, procedimientos y datos semilla.
SQL Server Management Studio	Herramienta recomendada en README para ejecutar el script de base de datos.
appsettings.json	Configuracion de cadena de conexion, JWT, expiracion y logging.

7. Instrucciones para ejecutar el proyecto correctamente

Prerrequisitos

- .NET SDK 8.0 o superior.
- SQL Server local.
- SQL Server Management Studio.

Configuración de base de datos

- Ejecutar Database/FondoXYZ.sql para crear la base FondoXYZ, tablas, restricciones y datos semilla.
- Ejecutar Database/SP_ConsultarHabitacionesDisponibles, Database/SP_ConsultarTarifas y Database/SP_CalcularTarifaReserva.
- Actualizar ConnectionStrings:FondoXYZ en appsettings.json con la instancia real de SQL Server.

Configuración de JWT

- Configurar Jwt:Issuer, Jwt:Audience y Jwt:Key. La aplicación no inicia si alguno falta.
- Jwt:ExpirationMinutes controla el tiempo de vida del token; si no es válido, el servicio usa 120 minutos.

Prueba de autenticación

- Enviar POST /api/auth/login con email maria.lopez@ejemplo.com o carlos.ruiz@ejemplo.com y clave 1234567.
- Usar el token devuelto en el encabezado Authorization: Bearer <token> para consumir endpoints protegidos.

Endpoints principales

Endpoint	Descripcion
POST /api/auth/login	Autentica asociado y devuelve JWT.
GET /api/sitios	Lista sitios activos.
GET /api/sitios/{sitioId}	Obtiene detalle de sitio, servicios, bloques y unidades.
GET /api/asociados	Lista asociados; acepta incluirInactivos=true.
POST /api/asociados	Crea asociado.
PUT /api/asociados/{asociadoId}	Actualiza asociado.
DELETE /api/asociados/{asociadoId}	Inactiva asociado, no lo elimina físicamente.
GET /api/disponibilidad	Consulta unidades disponibles por fechas, sitio y numero de personas.
GET /api/disponibilidad/alojamientos/{unidadAlojamientoId}	Consulta disponibilidad de una unidad específica.
POST /api/tarifas	Consulta tarifas vigentes.
POST /api/tarifas/calcular	Calcula total y detalle tarifario de una reserva.
POST /api/reservas	Crea reserva de alojamiento o visita de día.

POST /api/reservas/{reservald}/pagos	Registra pago; si es aprobado confirma la reserva.
--------------------------------------	--

8. Consideraciones finales

- La aplicación depende de datos semilla para sitios, unidades, tarifas y usuarios de prueba.
- La eliminación de asociados es lógica mediante Activo = false.
- Las rutas están protegidas por autenticación JWT; si se abre el navegador sin iniciar sesión puede parecer que la aplicación no responde correctamente.